GKE Configuration Reference

This document captures Google Kubernetes Engine (GKE) configuration patterns, best practices, and recommendations for the Graph OLAP Platform.

Prerequisites

Read these documents first:

- architectural.guardrails.md - Platform technology choices
- system.architecture.design.md - Overall architecture

Related Documents

- ryugraph-performance.reference.md - Pod memory configuration for RyuGraph
- ryugraph-wrapper.design.md - Wrapper pod specifications

Cluster Mode Selection

Standard vs Autopilot

From GKE modes of operation:

Factor	Standard	Autopilot
Node management	Manual	Fully managed
Pricing	Per node (VM)	Per pod (resources)
Max pods/node	256	32
Privileged containers	Yes	No
Custom node configs	Yes	Limited
Stateful workloads	**Recommended**	Possible
Control	Full	Limited

Recommendation: GKE Standard

For the Graph OLAP Platform, **GKE Standard** is recommended because:

1. **Stateful workloads**: RyuGraph instances are stateful with persistent volumes
2. **Custom node pools**: Need memory-optimized nodes (n2-highmem) with taints
3. **Pod density control**: Graph instances need guaranteed resources
4. **Privileged operations**: May need host-level access for debugging

```
# Cluster creation
gcloud container clusters create graph-olap-cluster \
  --mode=standard \
  --region=us-central1 \
  --release-channel=regular \
  --enable-ip-alias \
  --enable-private-nodes \
  --master-ipv4-cidr=172.16.0.0/28 \
  --workload-pool=PROJECT_ID.svc.id.goog
```

Networking

VPC-Native Clusters

From GKE networking best practices:

VPC-native clusters are **required** and provide:

- Pod-level firewall rules
- Direct VPC routing (no tunneling)
- Better performance
- Required for private clusters and Shared VPC

```
# VPC-native configuration
gcloud container clusters create graph-olap-cluster \
  --enable-ip-alias \
  --cluster-ipv4-cidr=/17 \
  --services-ipv4-cidr=/22
```

IP Address Planning

Resource	CIDR	Addresses	Notes
Nodes	/24	254	One per node
Pods	/17	32,768	~110 per node default
Services	/22	1,024	Cluster-wide
Master	/28	16	Control plane

Scaling consideration: Each new node requires its own IP + pod IP range. Plan for maximum cluster size.

Private Clusters

From GKE private clusters:

Private clusters provide:

- No public IPs on nodes
- Control plane accessible via private endpoint
- Reduced attack surface

```
# Private cluster configuration
gcloud container clusters create graph-olap-cluster \
  --enable-private-nodes \
  --enable-private-endpoint \
  --master-ipv4-cidr=172.16.0.0/28 \
  --enable-master-authorized-networks \
  --master-authorized-networks=10.0.0.0/8
```

Required for private clusters:

- Cloud NAT for outbound internet access
- Private Google Access for GCP APIs
- VPN/Interconnect for on-premises access

Cloud NAT Configuration

```
# Cloud NAT for private cluster egress
gcloud compute routers create graph-olap-router \
  --network=default \
  --region=us-central1
```

```
gcloud compute routers nats create graph-olap-nat \
  --router=graph-olap-router \
  --region=us-central1 \
  --nat-all-subnet-ip-ranges \
  --auto-allocate-nat-external-ips
```

Ingress and Load Balancing

Load Balancer Types

From GKE load balancing:

Type	Use Case	Annotation
External HTTP(S)	Public web traffic	`kubernetes.io/ingress.class: "gce"`
Internal HTTP(S)	Internal services	`kubernetes.io/ingress.class: "gce-internal"`
External TCP/UDP	Non-HTTP public	`cloud.google.com/load-balancer-type: "External"`
Internal TCP/UDP	Non-HTTP internal	`cloud.google.com/load-balancer-type: "Internal"`

Recommended: Gateway API

From GKE Gateway:

Gateway API is recommended for new deployments over Ingress:

```yaml
# Gateway for internal load balancing
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: graph-olap-gateway
spec:
  gatewayClassName: gke-l7-rilb  # Regional internal LB
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      tls:
```

```
        mode: Terminate
        certificateRefs:
          - name: graph-olap-cert
```

Control Plane Ingress Configuration

```
# Internal Ingress for Control Plane
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: control-plane-ingress
  annotations:
    kubernetes.io/ingress.class: "gce-internal"
    kubernetes.io/ingress.regional-static-ip-name: "control-plane-ip"
spec:
  tls:
    - hosts:
        - control-plane.internal
      secretName: control-plane-tls
  rules:
    - host: control-plane.internal
      http:
        paths:
          - path: /*
            pathType: ImplementationSpecific
            backend:
              service:
                name: control-plane
                port:
                  number: 8000
```

Graph Instance Routing

Graph instances are routed via path-based ingress:

```
# Dynamic path routing for instances
# Added by Control Plane when instance is created
- path: /{instance_id}/*
  pathType: ImplementationSpecific
  backend:
    service:
      name: graph-svc-{instance_id}
      port:
        number: 8000
```

Node Pools

Node Pool Strategy

Pool	Purpose	Machine Type	Taints
`default-pool`	System workloads	e2-standard-4	None
`control-plane-pool`	Control Plane	e2-standard-4	`workload=control-plane:NoSchedule`
`graph-instances-pool`	RyuGraph pods	n2-highmem-4	`workload=graph-instance:NoSchedule`

Graph Instances Node Pool

From ryugraph-performance.reference.md:

```
# Terraform configuration
resource "google_container_node_pool" "graph_instances" {
  name       = "graph-instances"
  cluster    = google_container_cluster.primary.name
  location   = "us-central1"

  initial_node_count = 1

  autoscaling {
    min_node_count = 0
    max_node_count = 10
  }

  node_config {
    machine_type = "n2-highmem-4"  # 4 vCPU, 32GB RAM
    disk_size_gb = 100
    disk_type    = "pd-ssd"

    # Workload Identity
    workload_metadata_config {
      mode = "GKE_METADATA"
    }

    # Isolate graph workloads
    taint {
      key    = "workload"
      value  = "graph-instance"
      effect = "NO_SCHEDULE"
    }
```

```
    labels = {
      "workload-type" = "graph-instance"
    }

    oauth_scopes = [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  }

  management {
    auto_repair  = true
    auto_upgrade = true
  }
}
```

Machine Type Selection

Machine Type	vCPU	Memory	Memory/CPU	Monthly Cost*	Use Case
e2-standard-4	4	16 GB	4 GB	~$97	General workloads
e2-highmem-4	4	32 GB	8 GB	~$135	Memory-intensive
n2-highmem-4	4	32 GB	8 GB	~$156	**Graph instances**
n2-highmem-8	8	64 GB	8 GB	~$312	Large graphs

*Approximate us-central1 pricing, on-demand

n2-highmem-4 recommended for graph instances due to:

- Higher memory bandwidth than e2 series
- Better single-thread performance
- Consistent performance (no CPU bursting)

Node Auto-Provisioning (Optional)

From GKE node auto-provisioning:

```
# Enable NAP for automatic node pool creation
gcloud container clusters update graph-olap-cluster \
```

```
  --enable-autoprovisioning \
  --min-cpu=0 --max-cpu=100 \
  --min-memory=0 --max-memory=1000 \
  --autoprovisioning-scopes=https://www.googleapis.com/auth/cloud-platform
```

Workload Identity

Overview

From Workload Identity Federation for GKE:

Workload Identity allows Kubernetes service accounts to act as IAM service accounts, eliminating the need for service account keys.

```
┌─────────────────────────────────────────────────────────────┐
│                        GKE Cluster                          │
│  ┌──────────────────────────────────────────────────────┐   │
│  │                        Pod                           │   │
│  │  K8s SA: graph-wrapper                               │   │
│  │      │                                               │   │
│  │      ▼                                               │   │
│  │  GKE Metadata Server                                 │   │
│  │      │                                               │   │
│  └──────┼───────────────────────────────────────────────┘   │
│         │                                                   │
│         ▼                                                   │
│  Workload Identity Pool                                     │
│  PROJECT_ID.svc.id.goog                                     │
└─────────┼───────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────────┐
│                          IAM                                │
│  GCP SA: graph-wrapper@PROJECT_ID.iam.gserviceaccount.com   │
│  Roles: storage.objectViewer, logging.logWriter             │
└─────────────────────────────────────────────────────────────┘
```

Configuration

Step 1: Enable on cluster

```
gcloud container clusters update graph-olap-cluster \
  --workload-pool=PROJECT_ID.svc.id.goog
```

Step 2: Create GCP service account

```
gcloud iam service-accounts create graph-wrapper \
  --display-name="Graph Wrapper Service Account"

# Grant permissions
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:graph-wrapper@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

Step 3: Create Kubernetes service account

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: graph-wrapper
  namespace: graph-olap
  annotations:
    iam.gke.io/gcp-service-account: graph-wrapper@PROJECT_ID.iam.gserviceaccount.com
```

Step 4: Bind accounts

```
gcloud iam service-accounts add-iam-policy-binding \
  graph-wrapper@PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:PROJECT_ID.svc.id.goog[graph-olap/graph-wrapper]"
```

Service Accounts Summary

K8s Service Account	GCP Service Account	Roles
`control-plane`	`control-plane@PROJECT.iam`	`cloudsql.client`, `pubsub.publisher`, `storage.admin`
`graph-wrapper`	`graph-wrapper@PROJECT.iam`	`storage.objectViewer`, `logging.logWriter`
`export-worker`	`export-worker@PROJECT.iam`	`storage.objectAdmin`, `cloudtasks.enqueuer`

Storage

StorageClass Configuration

From GKE persistent volumes:

Default StorageClasses (GKE 1.24+):

StorageClass	Disk Type	Replication	Use Case
`standard-rwo`	pd-balanced	Zonal	General workloads
`premium-rwo`	pd-ssd	Zonal	High IOPS
`standard-rwx`	Filestore	Regional	Shared access

Custom StorageClass for Graph Instances

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: graph-instance-ssd
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
  replication-type: none  # Zonal (single-zone)
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Delete
```

PersistentVolumeClaim for Graph Instance

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: graph-instance-{instance_id}-pvc
  namespace: graph-olap
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: graph-instance-ssd
  resources:
    requests:
      storage: 50Gi  # Adjust based on graph size
```

Storage Sizing Guidelines

Graph Size (edges)	Recommended PVC	Disk Type
< 1M	10Gi	pd-balanced
1M - 10M	50Gi	pd-ssd
10M - 100M	100Gi	pd-ssd
> 100M	200Gi+	pd-ssd

Note: RyuGraph uses disk for buffer pool spilling. Larger disks provide better I/O performance.

Resource Management

Namespace Resource Quotas

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: graph-olap-quota
  namespace: graph-olap
spec:
  hard:
    requests.cpu: "100"
    requests.memory: "200Gi"
    limits.cpu: "200"
    limits.memory: "400Gi"
    persistentvolumeclaims: "50"
    pods: "100"
```

LimitRange for Defaults

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: graph-olap-limits
  namespace: graph-olap
spec:
  limits:
    - type: Container
      default:
        cpu: "1000m"
        memory: "4Gi"
```

```
      defaultRequest:
        cpu: "500m"
        memory: "2Gi"
      max:
        cpu: "4000m"
        memory: "16Gi"
      min:
        cpu: "100m"
        memory: "256Mi"
```

Pod Resource Configuration

See ryugraph-performance.reference.md for detailed pod resource recommendations.

Summary:

```
# Graph Instance Pod
resources:
  requests:
    memory: "3Gi"
    cpu: "1000m"
  limits:
    memory: "8Gi"
    cpu: "2000m"

# Control Plane Pod
resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "2Gi"
    cpu: "1000m"
```

Observability

GKE Monitoring Integration

From GKE observability:

Enable all observability features:

```
gcloud container clusters update graph-olap-cluster \
  --enable-managed-prometheus \
```

```
  --logging=SYSTEM,WORKLOAD \
  --monitoring=SYSTEM,WORKLOAD
```

Managed Prometheus

From Google Cloud Managed Service for Prometheus:

Managed Prometheus is enabled by default on GKE. Configure scraping with PodMonitoring:

```yaml
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: control-plane-metrics
  namespace: graph-olap
spec:
  selector:
    matchLabels:
      app: control-plane
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
```

Key Metrics to Monitor

Cluster Level:

Metric	Query	Alert Threshold
Node CPU	`avg(rate(node_cpu_seconds_total{mode!="idle"}[5m]))`	> 80%
Node Memory	`node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`	< 20%
Pod Restarts	`increase(kube_pod_container_status_restarts_total[1h])`	> 5

Application Level:

Metric	Query	Alert Threshold
Request Latency	`histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))`	> 2s
Error Rate	`rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])`	> 1%
Graph Load Time	`graph_load_duration_seconds`	> 300s

Logging Configuration

```yaml
# Structured logging configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: logging-config
  namespace: graph-olap
data:
  LOG_FORMAT: json
  LOG_LEVEL: INFO
```

All application logs automatically flow to Cloud Logging when using structured JSON format.

Cost Optimization

Discount Strategies

From GKE cost optimization:

Strategy	Discount	Commitment	Best For
On-demand	0%	None	Variable workloads
Spot VMs	60-91%	None	Fault-tolerant batch
CUD 1-year	28-37%	1 year	Steady baseline
CUD 3-year	46-70%	3 years	Long-term stable

Recommended Approach

```
┌──────────────────────────────────────────────────────────┐
│                    Cost Optimization Strategy            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌─────────────────┐                                    │
│   │   Control Plane │  On-demand (always running, small) │
│   └─────────────────┘                                    │
│                                                          │
│   ┌─────────────────┐                                    │
│   │  Graph Instances│  CUD 1-year (predictable baseline) │
│   │    (baseline)   │  + On-demand (burst capacity)      │
│   └─────────────────┘                                    │
│                                                          │
│   ┌─────────────────┐                                    │
│   │  Export Workers │  Spot VMs (fault-tolerant, short-lived)│
│   └─────────────────┘                                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Spot VMs for Export Workers

Export Submitter and Poller Cloud Functions already provide cost efficiency through serverless pricing. For any batch processing workloads:

```
# Spot VM node pool for batch jobs
node_config {
  spot = true

  taint {
    key    = "cloud.google.com/gke-spot"
    value  = "true"
    effect = "NO_SCHEDULE"
  }
}
```

Right-Sizing with VPA

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: control-plane-vpa
  namespace: graph-olap
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
    name: control-plane
  updatePolicy:
    updateMode: "Off"  # Recommendation only
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed:
          cpu: "100m"
          memory: "256Mi"
        maxAllowed:
          cpu: "2000m"
          memory: "4Gi"
```

Security

Security Best Practices

From GKE security best practices:

Cluster Level:

- ☑ Use private clusters (no public node IPs)
- ☑ Enable Workload Identity (no service account keys)
- ☑ Enable Binary Authorization (optional, for image signing)
- ☑ Use VPC-native networking
- ☑ Enable master authorized networks

Workload Level:

- ☑ Run as non-root user
- ☑ Use read-only root filesystem where possible
- ☑ Drop all capabilities, add only required
- ☑ Use network policies

Pod Security Standards

```
apiVersion: v1
kind: Pod
metadata:
  name: graph-instance-{id}
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
```

```yaml
    fsGroup: 1000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: wrapper
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: false  # RyuGraph needs write access
        capabilities:
          drop:
            - ALL
```

Network Policies

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: graph-instance-policy
  namespace: graph-olap
spec:
  podSelector:
    matchLabels:
      app: graph-instance
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Allow from ingress controller
    - from:
        - namespaceSelector:
            matchLabels:
              name: ingress-nginx
      ports:
        - port: 8000
    # Allow from control plane
    - from:
        - podSelector:
            matchLabels:
              app: control-plane
      ports:
        - port: 8000
  egress:
    # Allow to GCS (via Private Google Access)
    - to:
        - ipBlock:
            cidr: 199.36.153.8/30  # restricted.googleapis.com
      ports:
        - port: 443
    # Allow DNS
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
```

```
        - port: 53
          protocol: UDP
```

Cluster Maintenance

Release Channels

Channel	Updates	Stability	Recommendation
Rapid	Frequent	Lower	Development
Regular	Monthly	Balanced	**Production**
Stable	Quarterly	Highest	Regulated environments

```
gcloud container clusters update graph-olap-cluster \
  --release-channel=regular
```

Maintenance Windows

```
gcloud container clusters update graph-olap-cluster \
  --maintenance-window-start=2024-01-01T02:00:00Z \
  --maintenance-window-end=2024-01-01T06:00:00Z \
  --maintenance-window-recurrence="FREQ=WEEKLY;BYDAY=SA,SU"
```

Pod Disruption Budgets

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: control-plane-pdb
  namespace: graph-olap
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: control-plane
```

Quick Reference

Essential Commands

```
# Get cluster credentials
gcloud container clusters get-credentials graph-olap-cluster --region us-central1

# Check node allocatable resources
kubectl get nodes -o custom-columns=\
NAME:.metadata.name,\
CPU:.status.allocatable.cpu,\
MEMORY:.status.allocatable.memory

# Check pod resource usage
kubectl top pods -n graph-olap

# Check node pool status
gcloud container node-pools list --cluster graph-olap-cluster

# Scale node pool
gcloud container clusters resize graph-olap-cluster \
  --node-pool graph-instances \
  --num-nodes 5
```

Environment Configuration

Environment	Cluster	Node Pool Min/Max	CUD
Development	Standard, public	0-3	None
Staging	Standard, private	1-5	None
Production	Standard, private	2-10	1-year

References

Google Cloud Documentation

- GKE Best Practices - Networking
- Workload Identity Federation
- GKE Persistent Volumes
- GKE Observability

- Managed Service for Prometheus
- GKE Cost Optimization

Feature Comparison

- Autopilot vs Standard
- Node Pool Configuration
- Load Balancing Options